

School Management App

Fees Modul – Speciális Akciók és Route Sorrend

React 18 + Laravel 12 API | 2026

1. A Fees modul felépítése

A Fees (Díjak) modul a sima CRUD mintát (FeeListPage, FeeFormPage) egészíti ki két speciális elemmel: egy AKCIÓ gombbal (befizetés jelölése, nem sima update) és egy KÜLÖN SZŰRT nézettel (lejárt díjak, saját route-tal és queryKey-vel).

2. markAsPaid – akció végpont, nem sima update

A díj 'befizetettnek' jelölése SZÁNDÉKOSAN nem egy sima PUT /fees/{id} {status: 'paid'} hívás, hanem egy dedikált POST /fees/{id}/pay végpont.

```
export function useMarkFeeAsPaid() {
  const queryClient = useQueryClient();

  return useMutation({
    mutationFn: (id) => api.post(`/fees/${id}/pay`).then((res) =>
      res.data),
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ['fees'] });
    },
  });
}

// Használat - csak az id-t kell átadni, semmi mást:
await markAsPaid.mutateAsync(fee.id);
```

Miért nem elég egy sima update({status: 'paid'})?

Mert a befizetés jelölése TÖBB mezőt is módosít egyszerre, amiket a frontendnek NEM kellene ismernie vagy kezelnie: a backend FeeService::markAsPaid() automatikusan beállítja a paid_date-et (mai dátum) ÉS legenerál egy egyedi receipt_number-t (bizonylatszámot, pl. 'RCP-2026-000042'). Ha ezt egy sima update()-tel próbálnánk megoldani, a frontendnek magának kellene kiszámolnia a bizonylatszámot - ami üzleti logika, és NEM a frontend felelőssége. A dedikált végpont (POST /fees/{id}/pay) tisztán elválasztja: a frontend csak JELZI a szándékot ('fizessük ki ezt'), a backend dönti el HOGYAN kell ezt csinálni.

3. Feltételes gomb megjelenítés a listában

```
{fee.status !== 'paid' && (  
  <button onClick={() => handleMarkAsPaid(fee.id)}>  
    <i className='bi bi-check-lg'></i>  
  </button>  
)}
```

Miért csak akkor jelenik meg a gomb, ha status !== 'paid'?

Ha egy díj MÁR be van fizetve, a 'Befizetve' gomb megjelenítése félrevezető lenne - a felhasználó azt hihetné hogy még tennie kell valamit. A feltételes renderelés (fee.status !== 'paid' &&) csak azoknál a díjaknál mutatja a gombot, ahol ténylegesen VAN mit tenni (pending, overdue, vagy akár waived esetén is - bár ez utóbbi eset üzleti döntés kérdése, jelenleg megengedjük).

4. is_overdue – backend számított mező megjelenítése

```
<td>  
  {fee.due_date}  
  {fee.is_overdue && (  
    <span className='badge bg-danger ms-2'>Lejárt</span>  
  )}  
</td>
```

Honnan jön az is_overdue mező?

Ez a FeeResource.php-ban számított mező, nem az adatbázisból jön közvetlenül: 'is_overdue' => \$this->status === 'pending' && \$this->due_date->isPast(). A frontend egyszerűen MEGJELENÍTI ezt a már kiszámított boolean értéket - nem kell neki saját maga összehasonlítani dátumokat vagy figyelnie a mai napot. Ez a Resource réteg egyik előnye: az üzleti logika (mi számít lejártnak) egy helyen van definiálva, a frontend csak fogyasztja az eredményt.

5. KRITIKUS – route sorrend a statikus és dinamikus útvonalaknál

Az AppRoutes.jsx-ben a /fees/overdue route-nak MEGELŐZI kell a /fees/:id/edit route-ot, különben a React Router hibásan próbálná az 'overdue' szót id-ként értelmezni.

```
// HELYES sorrend:  
<Route path='/fees' element={<FeeListPage />} />
```

```

<Route path='/fees/create' element={<FeeFormPage />} />
<Route path='/fees/overdue' element={<OverdueFeesPage />} /> // ELŐBB!
<Route path='/fees/:id/edit' element={<FeeFormPage />} /> // UTÁNA

// HIBÁS sorrend (elméleti példa - ne így csináld):
<Route path='/fees/:id/edit' element={<FeeFormPage />} /> // ELŐBB
<Route path='/fees/overdue' element={<OverdueFeesPage />} /> // sose éri el!

```

Miért számít a sorrend a React Router-ben?

A React Router (v6) a route-okat a definiált SORRENDEN próbálja illeszteni az URL-hez, és az ELSŐ egyezésnél megáll. A `:id` egy dinamikus paraméter, ami BÁRMILYEN szöveget elfogad - beleértve az `'overdue'` szót is! Ha a `/fees/:id/edit` route korábban szerepelne, a `/fees/overdue` URL-re navigálva a React Router azt hinné hogy `id='overdue'`, és megpróbálná a `FeeFormPage`-et megnyitni egy `'overdue'` nevű (nem létező) fee id-val, ami hibához vezetne. A statikus route-okat (fix szöveggel, mint `/overdue` vagy `/create`) MINDIG a dinamikus (`:id` mintás) route-ok ELÉ kell tenni.

6. Külön queryKey a szűrt nézethez

```

// Teljes lista
export function useFees(page = 1) {
  return useQuery({ queryKey: ['fees', page], ... });
}

// Csak a lejárt díjak - KÜLÖN queryKey
export function useOverdueFees() {
  return useQuery({ queryKey: ['fees', 'overdue'], ... });
}

```

Miért kell külön queryKey a szűrt nézetnek?

Ha mindkét lekérdezés ugyanazt a `['fees']` kulcsot használná, a React Query cache-ében az EGYIK felülírná a MÁSIKAT - hol a teljes lista, hol csak a lejártak lennének a cache-ben, attól függően melyiket kértük le utoljára. A `['fees', 'overdue']` egy KÜLÖNÁLLÓ cache bejegyzés, ami egymástól függetlenül tárolja a két, eltérő tartalmú listát. Fontos ugyanakkor hogy mindkettő `'fees'`-szel KEZDŐDIK, így egy `invalidateQueries({queryKey: ['fees']})` mindkettőt érvényteleníti egyszerre - erre van szükség, mert egy befizetés jelölése után MINDKÉT nézetnek frissülnie kell.

7. Visszatérő hiba – harmadik előfordulás

#	Modul	Elgépelés	Helyes forma
---	-------	-----------	--------------

1.	Subjects	exists:teacher,id	exists:teachers,id
2.	Classes	(korábban javítva a TeacherResource id hiánya kapcsán)	—
3.	Fees	exists:student,id	exists:students,id

Miért érdemes ez alapján egy ellenőrző szokást kialakítani?

Ez már a MÁSODIK konkrét exists:X,id elgépelés (egyes szám vs többes szám tábla névnel) ugyanabban a projektben. Ez egy mintázat: minden ÚJ Form Request írásakor, mielőtt tesztelnéd a frontendet, érdemes VÉGIGNÉZNI az összes exists: és unique: szabályt, és ellenőrizni hogy a táblanevek helyesen, TÖBBES SZÁMBAN szerepelnek-e (students, teachers, classes, subjects, exams, fees) - ez egy 10 másodperces ellenőrzés ami megspórol egy 500-as hibával járó debug kört.

– School Management App fejlesztési dokumentáció –